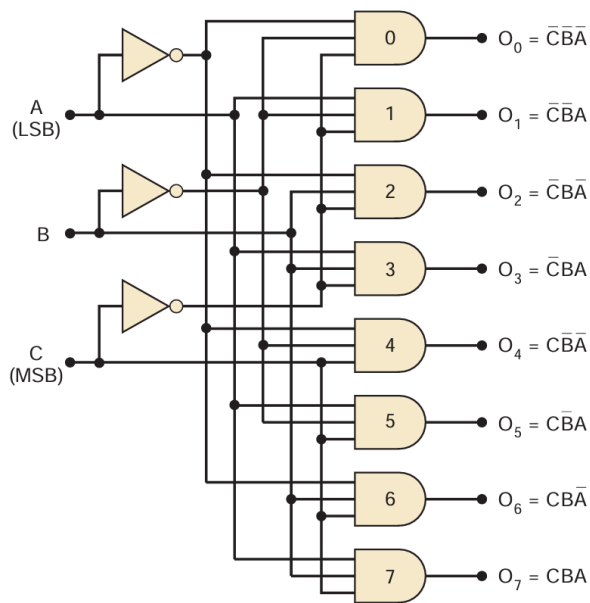


Design: 4-to-16-line Decoder



C	B	A	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

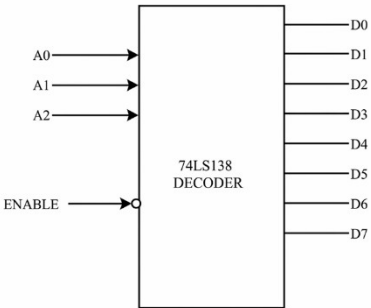


Fig - 3X8 Decoder

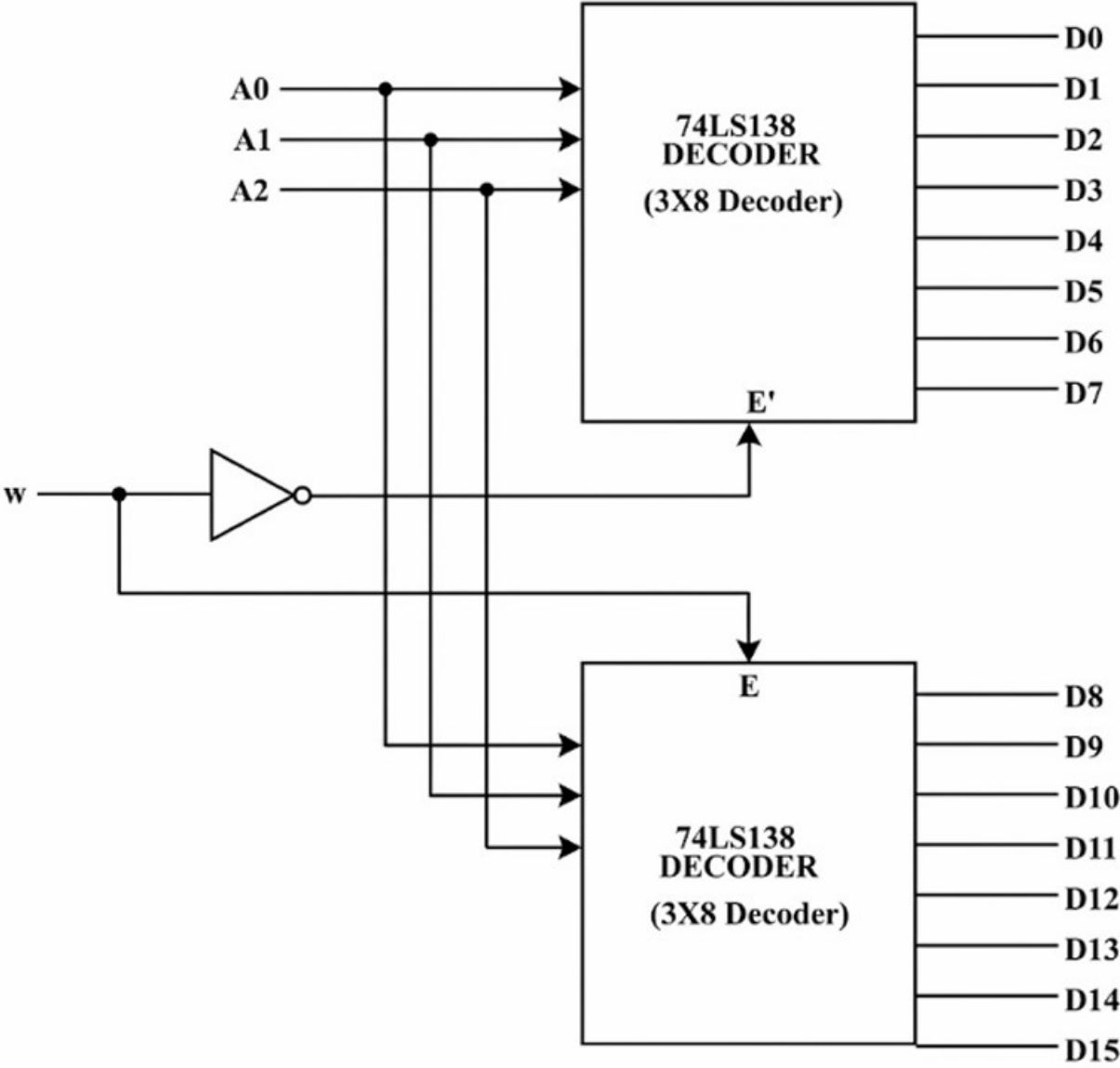


Fig - 4X16 Decoder

```

module decoder_3to8 (
input wire [2:0] sel,
input wire en,
output wire [7:0] y);

wire A,B,C;

assign A = sel[0];
assign B = sel[1];
assign C = sel[2];

//Active low
assign y[0] = (~en & ((~C)&(~B)&(~A)) );
assign y[1] = (~en & ((~C)&(~B)&(A)) );
assign y[2] = (~en & ((~C)&(B)&(~A)) );
assign y[3] = (~en & ((~C)&(B)&(A)) );
assign y[4] = (~en & ((C)&(~B)&(~A)) );
assign y[5] = (~en & ((C)&(~B)&(A)) );
assign y[6] = (~en & ((C)&(B)&(~A)) );
assign y[7] = (~en & ((C)&(B)&(A)) );

endmodule

module decoder_4to16 (
input wire [3:0] sel,
output wire [15:0] y
);

wire en;
assign en = sel[3];

decoder_3to8 one (.sel(sel[2:0]), .en(en), .y(y[7:0]));
decoder_3to8 two (.sel(sel[2:0]), .en(~en), .y(y[15:8]));

endmodule

```

```

`timescale 1ns/1ps

module tb_decoder_4to16;

    reg [3:0] sel;
    wire [15:0] y;
    integer i;

    decoder_4to16 uut (
        .sel(sel),
        .y(y)
    );

    initial begin
        $display("sel    y");
        $monitor("%b    %b", sel, y);

        for (i = 0; i < 16; i = i + 1) begin
            sel = i;
            #10;
        end

        $finish;
    end

endmodule

```

sel	y
0000	0000000000000001
0001	0000000000000010
0010	0000000000000100
0011	0000000000001000
0100	0000000000010000
0101	0000000000100000
0110	0000000001000000
0111	0000000010000000
1000	0000000100000000
1001	0000001000000000
1010	0000010000000000
1011	0000100000000000
1100	0001000000000000
1101	0010000000000000
1110	0100000000000000
1111	1000000000000000